Please take

The Bonus of Course

Bonus book has

- 28 Topics
- 98 pages

Contents

0. How to use this course
1. Introduction to Programming
2. Installing Anaconda
3. Installing and Using Jupyter-Notebook
4. Print function
5. Variables in Python
6. Space and Tab errors
7. Python Data Types
8. SyntaxError and NameError
9. Data Conversion
10. Comments in Python
11. Booleans in Python
12. Python Operators
13. Python Statements
14. Python Loops
15. Python Arrays
16. Python Strings
17. Python Lists
18. Python Tuples
19. Python Sets
20. Python Dictionary
21. Python Functions
22. Python Lambda
23. Input function
24. Python Classes and Objects
25. Python Inheritance
26. Python Scopes
27. Python Error Handling
28. Python Modules

1 Introduction to Programming

Introduction to

- Program & Programming
- Programming Languages
- Python Programming Language
- Python IDEs

Program meaning

Program is also called

Software

Application

which is **created** by **programming**
for a machine to **perform** specified tasks

Examples for programs:

Google

facebook

Instagram

Programming meaning

Programming **is** the **process** or **activity** of **writing programs** for **computers** or **any kind of devices**

We write **programs** by **Programming Languages**

There are many kind of Programming Languages:

However we are going to learn Python Programming Language in this course

Python IDEs

What is **IDE**?

IDE is the short form of Integrated Development Environment.

IDEs are coding tools which are used to write codes.

The **most popular IDEs** are for **writing Python codes**:

IDLE

Jupyter Notebook

Spyder

Atom

Visual Studio Code

PyCharm

However in **this course**, we will use **Jupyter Notebook** and **PyCharm**

Anaconda Installation

If you want to download Anaconda for a different **OS** (e.g. **Mac** or **Linux**), go **the down** of download link, and you can choose Anaconda for a different **OS** as the bellow image:

My **OS** is **Windows**, therefore I choose Anaconda for Windows (64-Bit Graphical Installer).

Anaconda Download List:

Windows:

64-Bit Graphical Installer

32-Bit Graphical Installer

MacOS:

64-Bit Graphical Installer

64-Bit Command Line Installer

Linux:

64-Bit (x86) Installer (544 MB)

64-Bit (Power8 and Power9) Installer (285 MB)

Initializing Python

Write “python” to initialize **Python** on Anaconda Prompt:

Anaconda Prompt (anaconda3) - python

```
(base) C:\Users\sabr>python
Python 3.8.8 (default, Apr 13 2021, 15:08:03) [MSC v.1916 64 bit (AMD64
)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print('Hello World')
Hello World
>>> print('Hello Python, I can learn you:)')
Hello Python, I can learn you:)
>>>
```

3 Installing and Using Jupyter-Notebook

Jupyter-notebook: Installation

You can install jupyter by using **conda** or **pip. conda** and **pip** are used to installing **python packages (libraries)**

by **conda** →

```
conda install -c anaconda jupyter
```

by **pip** →

```
pip install jupyter
```

Note:

Jupyter-notebook should be installed automatically, when you **installed Anaconda**.
If it was not installed automatically, you can use above codes for installing jupyter-notebook

4 Print function

Print function: print(*object(s)*)

```
print(object(s))
```

object can be a **number**

Ex 1)

```
print(5)
```

```
5
```

objects can be **many numbers** (*sequence of numbers*). Separate each **number** with comma **,**

Ex 2)

```
print(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

```
0 1 2 3 4 5 6 7 8 9 10
```

objects can be *sequence of* **texts numbers.** Separate each item (text or number) with comma **,**

Ex 3)

```
print(0, 'Hello', 1, 'I', 'am', 2, 'am', 3, 'python', 4, 5, 6, 7, 8, 9, 10)
```

```
0 Hello 1 I am 2 am 3 python 4 5 6 7 8 9 10
```

5 Variables in Python

Variables in Python

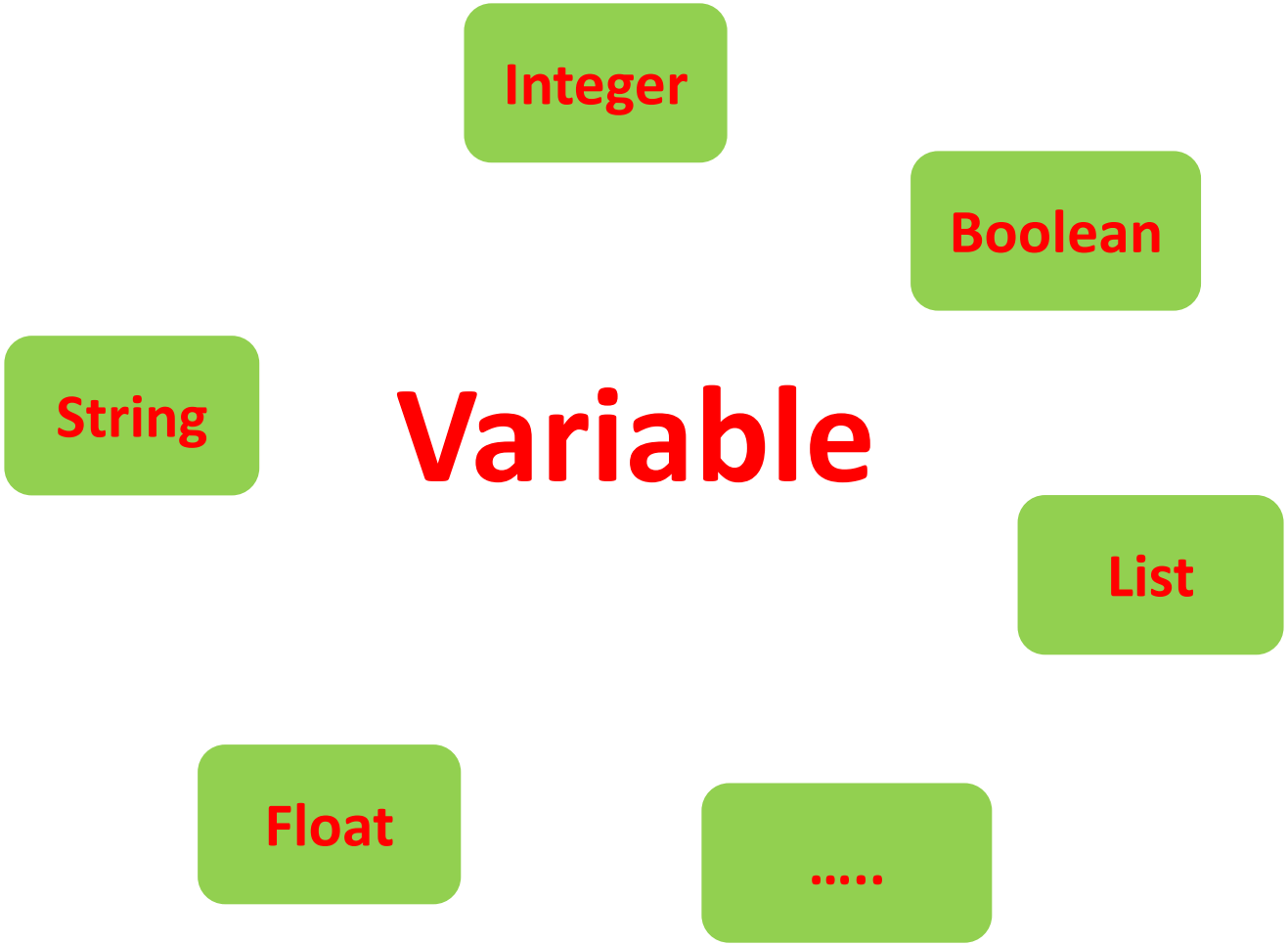
What is Variable?

Variable is a **container** for **storing data values**.

Variable =

Variables in Python

There are **many kind** of **Variables** in Python



Variables in Python: type() function

```
a = str(7)
```

Variable name

str() function

Integer value

type() function check the **type** of the Variable

```
print('data type',type(a))
```

```
data type <class 'str'>
```

Variable type (class) is String

Variables in Python

We have learnt

```
string_variable = 'Python'

integer_variable = 3

float_variable = 3.8
```

In future lessons,
We will learn

```
complex_variable = 3 + 2j

boolean_variable = True

list_variable = ['Python', 3, 3.8]

tuple_variable = (3, 'Python', 3.8)

set_variable = {3.8, 'Python', 3}

dictionary_variable = {'name':'Python', 'version':3}
```

Variables in Python

We have learnt

```
string_variable = 'Python'

integer_variable = 3

float_variable = 3.8
```

In future lessons,
We will learn

```
complex_variable = 3 + 2j

boolean_variable = True

list_variable = ['Python', 3, 3.8]

tuple_variable = (3, 'Python', 3.8)

set_variable = {3.8, 'Python', 3}

dictionary_variable = {'name':'Python', 'version':3}
```

6 Space Error and Tab Error

Space Error and Tab Error in Python

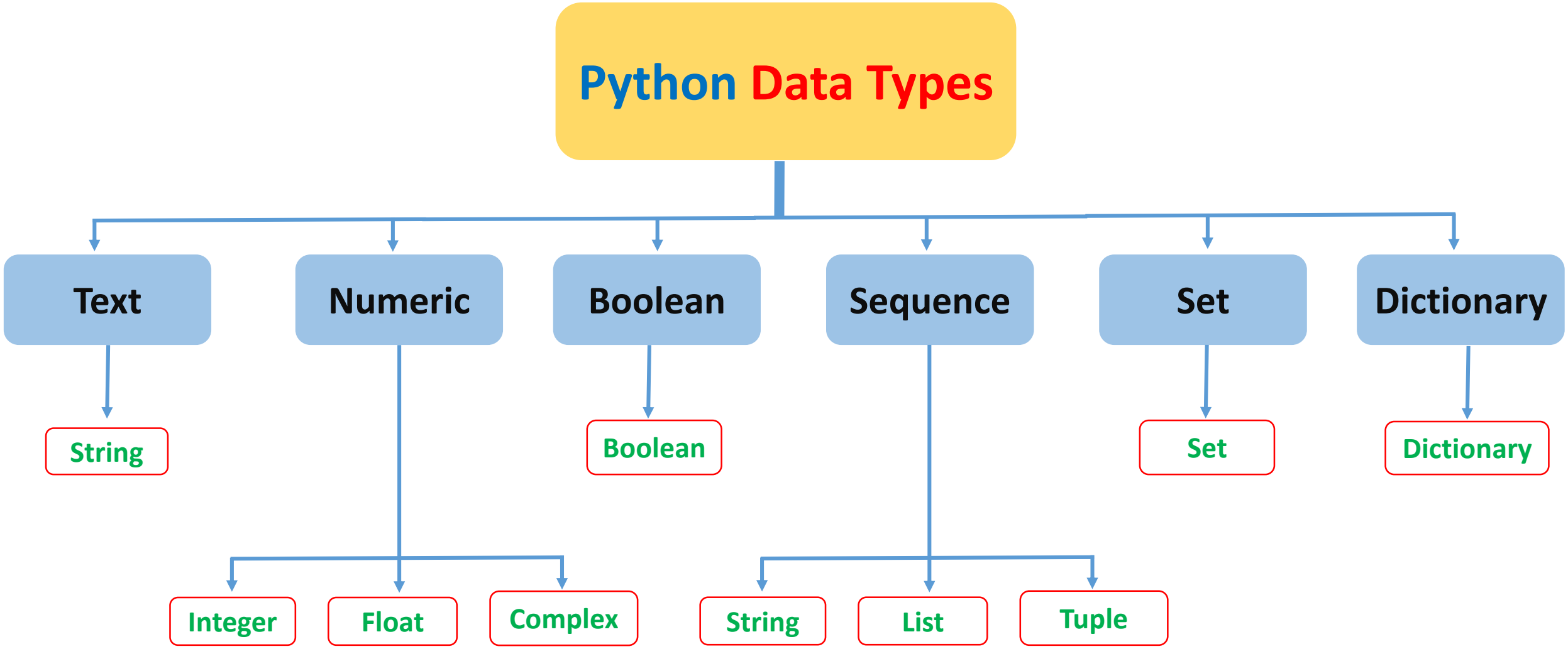
```
name = 'Jonny'
print(name)
```

```
Jonny
```

We have to use the same number of spaces or same number of tabs in the same block of code, otherwise Python will give you an **error**:

7 Python Data Types

Python Data Types: Introduction



Text Type: Single-line String

String Variable Value should be **in 'Single Quotation'** or **in "Double Quotation"** or **in str() function**

Single Quotations:

```
name_sq = 'Python'
print(name_sq)
```

```
Python
```

Single-line text

Double Quotations:

```
name_dq = "Python"
print(name_dq)
```

```
Python
```

Single-line text

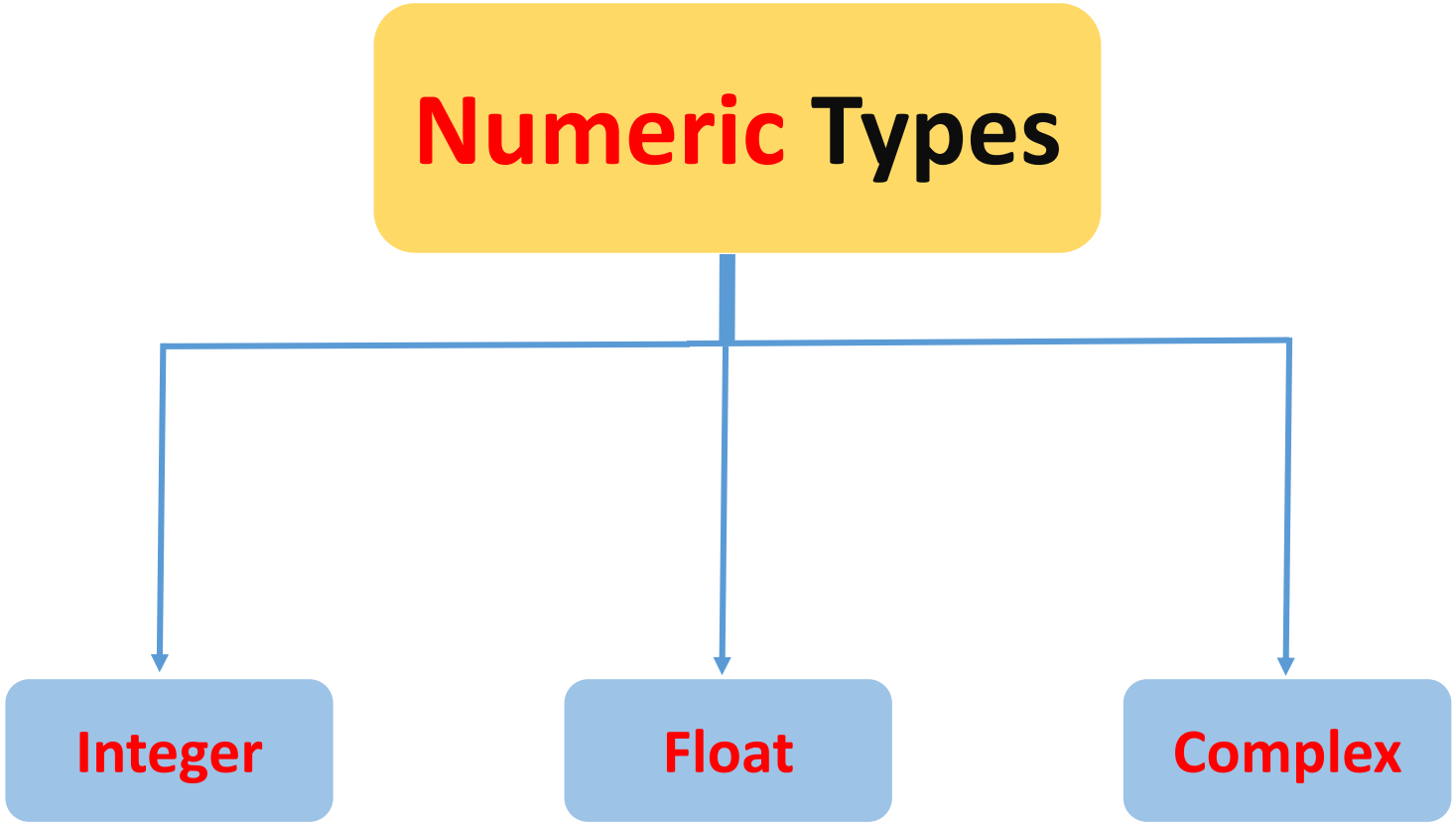
str() function:

```
age = str(25)
print(age)
```

```
25
```

Single-line text

Numeric Types: Integer, Float, and Complex



Numeric Types: Float

Or we can you use float() function for creating float **variables** value from string or integer **variables**.

But, if you want to **convert** string **variable to** float **variables:**

the **value** of string **variable must be number.**

Check Variable type by type() function

String **to** Float

```
age = '25'

age = float(age)
print(age)
```

```
25.0
```

```
print(type(age))
```

```
<class 'float'>
```

Integer **to** Float

```
number = 22

number = float(number)
print(number)
```

```
22.0
```

```
print(type(number))
```

```
<class 'float'>
```

Numeric Types: Complex

What is Complex number?

Complex number is similar to integer or float number, but it has the "j" after number at imaginary part.

Complex numbers are written with a "j" as the imaginary part:

Example:

Declaring complex variable

"j" character changes number to complex number

```
x = 1j
```

Complex variable name

number

Note: "j" must be written after number.
There is no any space between "j" and number

8 SyntaxError and NameError in Code

NameError in Code

If you **assign** undeclared (undefined) variable as the **value** of other **variable**, you will get **NameError** :

Ex 1) undefined (b) variable as the **value** of new **variable**

```
a = b

print(a)
```

```
---------------------------------
NameError
<ipython-input-1-ed99eac7bf8d> in <
----> 1 a = b
      2
      3 print(a)

NameError: name 'b' is not defined
```

Ex 2) undefined (b) variable as the **value** of new **variable**

```
b = a

print(b)
```

```
---------------------------------
NameError
<ipython-input-1-50082f0d24ed> in
----> 1 b = a
      2
      3 print(b)

NameError: name 'a' is not defined
```

9 Data Conversion

Data Conversion in Python

Integer
Float
Complex

to **String**

String
Integer
Complex

to **Float**

String
Float
Complex

to **Integer**

String
Integer
Float

to **Complex**

Data Conversion in Python: Convert to Float

Convert String, Integer, Complex **to** **Float by float() function**

You **can** convert **String** to Float by using float() function, if **string variable value** is **number** or **decimal number**.

String to Float

```
my_str = '10'

my_float = float(my_str)
print(my_float)
print(type(my_float))
```

```
10.0
<class 'float'>
```

String to Float

```
my_str = '10a'

my_float = float(my_str)
print(my_float)
print(type(my_float))
```

```
-----------------------
ValueError
```

Wrong conversion, because **string** value is not only number

10 Comments in Python

Comments in Python

Comments are used to **explain Python code**.

Comments are used to **make the code more readable**.

To make a comment, we can use bellow symbols:

1) Hash

```
# Text
```

Hash # is for a ***single-line comment***

2) Three single quotations

```
'''
Text
'''
```

3) Three double quotations

```
"""
Text
"""
```

Triple Quotes is for ***multi-line comments***

Note: **Hash #** is preferred for a ***single-line or for 2, or 3 line comments***. However, Triple Quotes is preferred for ***multi-line comments***

11 Booleans in Python

Booleans in Python

There are **2 types of Boolean values** in Python

True

```
my_bool = True

print(my_bool)
print(type(my_bool))
```

```
True
<class 'bool'>
```

False

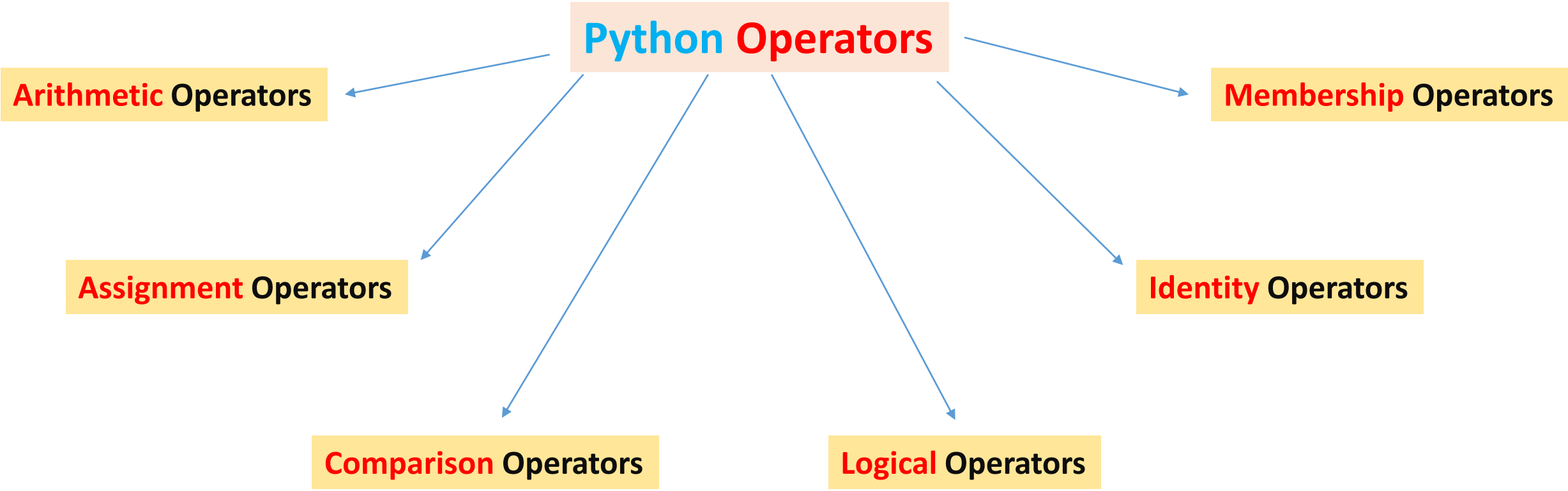
```
my_bool = False

print(my_bool)
print(type(my_bool))
```

```
False
<class 'bool'>
```

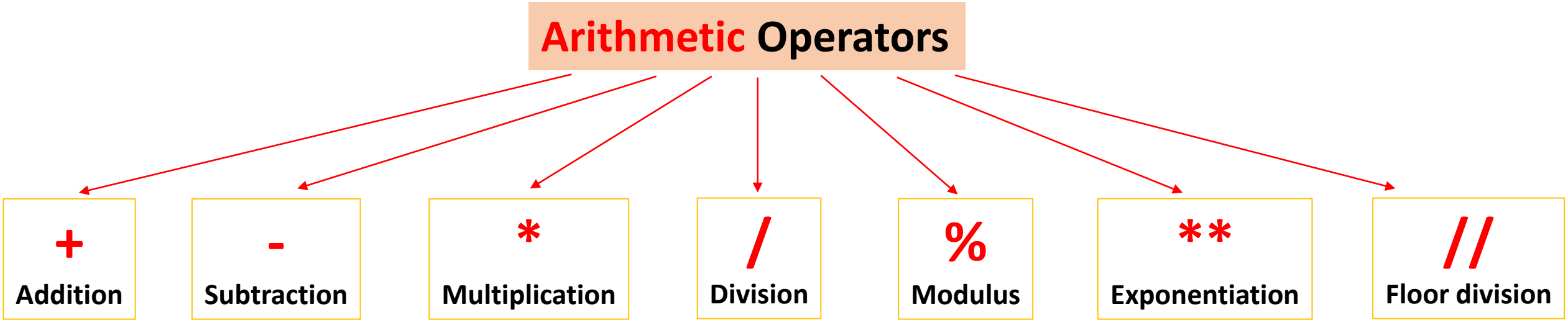
12 Python Operators

Python Operators



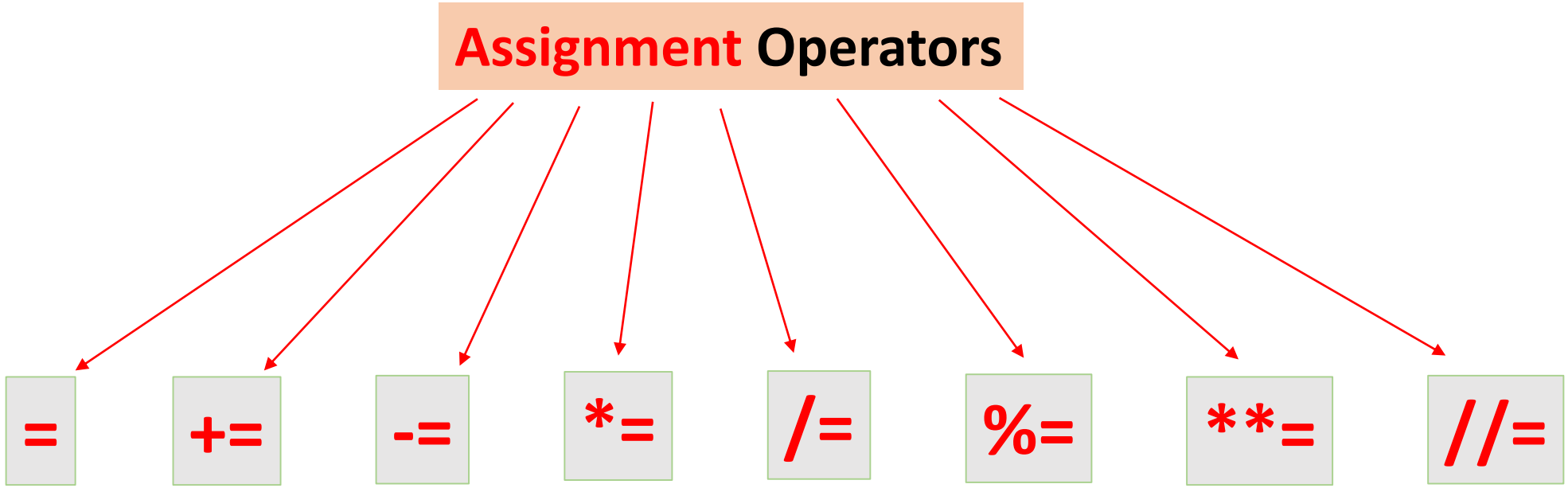
Operators can be used to perform operations on variables and values.

Python Operators: Arithmetic Operators



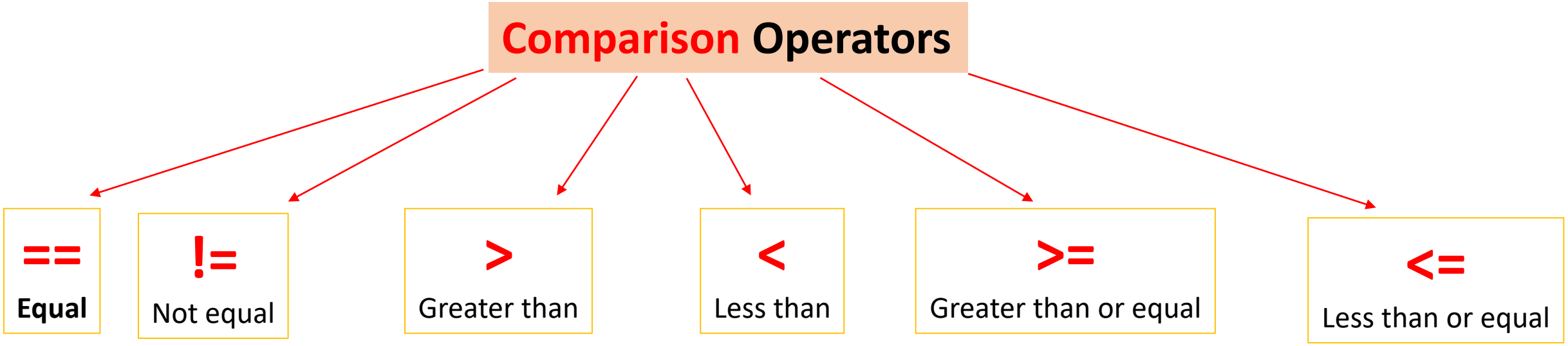
We use **Arithmetic operators** with **numeric values** to **perform** common **mathematical operations**.

Python Operators: Assignment Operators



Assignment operators is used to ***assign values*** ***to variables***

Python Operators: Comparison Operators



We use **Comparison operators** to **compare** two values or variables.

Note: The result of Comparison will be equal to **Boolean value** (True or False)

Python Operators: Logical Operators

Logical Operators

- **and** → Returns **True** if all condition statements are true
- **or** → Returns **True** if one of all condition statements is true
- **not** → **Reverse the result**, returns **False** if the result is true

We use **Logical operators** to **combine two** or **many** **conditional statements.**

Note: The result of Logical operators will be equal to **Boolean value** (**True** or **False**)

Python Operators: Identity Operators

Identity Operators

- **is** → Returns **True** if both variables are the same object, else returns **False**
- **is not** → Returns **True** if both variables are not the same object, else returns **False**

Identity operators are used to compare the **variables**, not if they are equal, but if they are actually the same object, with the **same memory** location.

Note: The result of Identity operators will be equal to **Boolean value** (**True** or **False**)

Python Operators: Membership Operators

Membership Operators

- **in** → Returns **True** if a sequence with the **specified value** is present in the object, else returns **False**
- **not in** → Returns **True** if a sequence with the **specified value** is not present in the object, else returns **False**

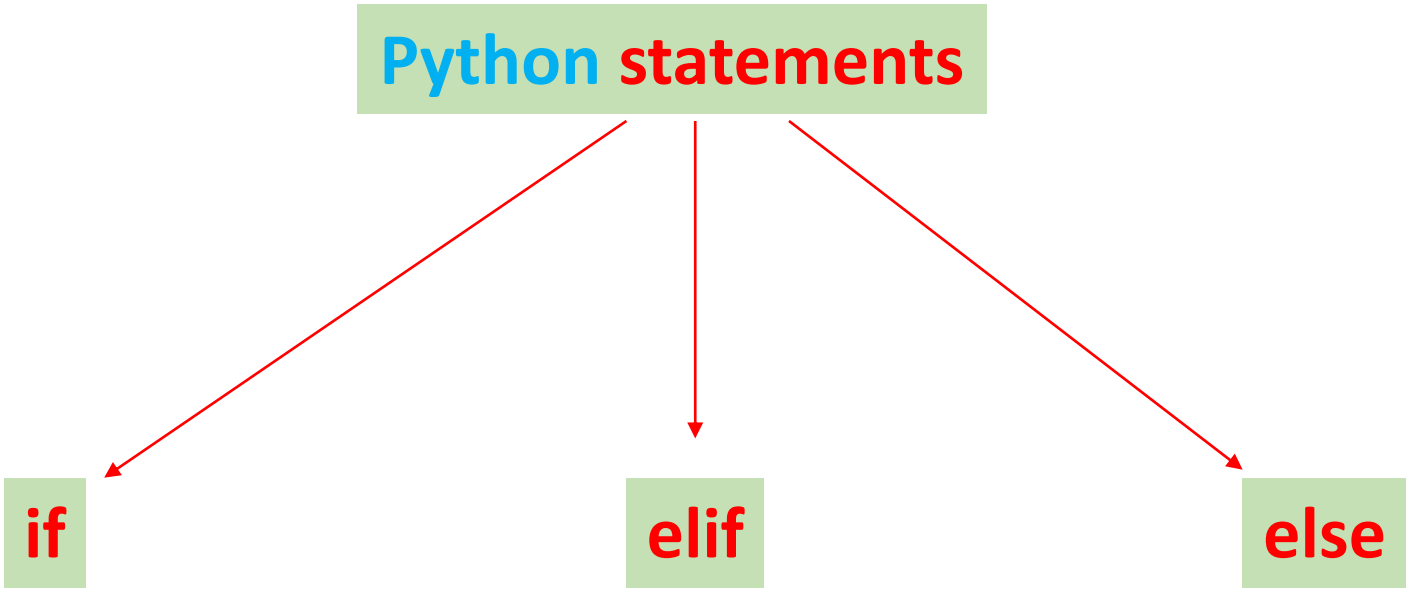
Membership operators are used to **check** if a **sequence** is **presented** in an **object**

.

Note: The result of Membership operators will be equal to **Boolean value** (**True** or **False**)

13 Python Statements

Python statements



If statement

If Boolean value is **False**, the code inside of "**if statement**" doesn't **work**.

Comparing two values. If Comparison is **False**, the **"Correct" will not be printed:**

```
if 10 < 9:
    print("Correct")
```

if statement

```
if 10 < 9:
    print("Correct")
```

Whitespace: **a Tab** (Indentation)

If statement: pass

if statement cannot be empty, however if you don't want this statement to return to any output, just put pass **inside** if statement.
pass statement passes if statement, and it avoids getting error.

Comparison is True

```
if 10 > 5:
    pass
```

pass statement passes if statement.
it avoids getting error.

No output, because
of pass statement

14 Python Loops

Python Loops

There are **two** types of Loops in **Python**:

while

for

For Loop: The break statement

The **break** statement can **stop** "**the for** loop" before "the **for** loop" **looped** through all the items

```
my_numbers = "123456789"

for n in my_numbers:
    print(n)
    if n == '5':
        break
```

break stopped the loop if n == '5'

```
1
2
3
4
5
```

The output is printed until n==5.

For loop: range() - specific increment value

The range() function the default increment value is **1**, however it is possible to change the increment value by adding a third parameter: range(0, 10, **2**):

```
for a in range(0, 10, 2):
  print(a)
```

1st parameter | 2nd parameter | 3rd parameter

outputs

```
0
2
4
6
8
```

You used *2 as* **the specific increment value** on the 3rd parameter of range() function, therefore outputs are 0, 2, 4, 6, 8

15 Python Arrays

Arrays

What is an Array?

If the variable **holds more than one value at a time,** it is called the Array.

The groups of Arrays in Python: **String, List, Tuple, Set, Dictionary**

Sequence types

Set type

Dictionary type

```
str_array = 'Python'

list_array = [1, 2, 3, 4]

tuple_array = (1, 2, 3, 4)

set_array = {1, 2, 3, 4}

dictionary_array = {'one':1, 'two':2, 'three':3, 'four':4}
```

Note: Python does not have built-in support for Arrays, but **Python Lists** can be used instead Arrays

We will learn all of them one by one.

16 Python Strings

Python Strings: Index Order

Note: **String element** starts **from 0** ***not 1***

1st = 0
2nd = 1
3rd = 2
...

```
str_variable = 'Python'
```

Index Order

0	1	2	3	4	5
1st	2nd	3rd	4th	5th	6th

If you want to print **"P"** from string variable:

```
print(str_variable[0])
```

```
P
```

If you want to print **"h"** from string variable:

```
print(str_variable[3])
```

```
h
```

17 Python Lists

Python Lists

Lists can store **multi-different types of values** in a single variable.

Integer Float String Boolean

```
list_variable = [1, 2.0, "three", True]

print(list_variable)
```

```
[1, 2.0, 'three', True]
```

Python Lists: Items, Negative Indexing

Negative indexing is to **start** from the end.

-1 is **1st last** item of **List**, -2 is **2nd last** item of **List** and so on

```
list_variable = [1, 2.0, "three", True]
```

-4 -3 -2 -1

Print **1st last** item of **list_variable**:

```
list_variable = [1, 2.0, "three", True]

print(list_variable[-1])
```

```
True
```

Python Lists: Looping Using List Comprehension

List Comprehension is the shortest syntax to loop through lists.

```
list_variable = ['Alex', 'John', 'Anna']
[print(a) for a in list_variable]
```

Everything in one line

```
Alex
John
Anna
```

With **list comprehension**
you can do all **in one line of code**

```
[print(a) for a in list_variable]
```

=

Without list comprehension
you must write 2 line code

```
for a in list_variable:
    print(a)
```

18 Python Tuples

Python Tuples: Items, Index order

Tuple **items** are **indexed**, **1st** item is index [0], **2nd** item is index [1] and so on

```
tuple_variable = (1, 2.0, "three", True)
```

0 1 2 3

Print **1st** item of **tuple_variable**:

```
tuple_variable = (1, 2.0, "three", True)

print(tuple_variable[0])
```

```
1
```

Print **3rd** item of **tuple_variable** :

```
tuple_variable = (1, 2.0, "three", True)

print(tuple_variable[2])
```

```
three
```

Python Lists: len() function

To know **how many items** a **tuple** has, use the **len()** function.
len() function shows **number of items of your tuple**

The **name** of Tuple variable

```
tuple_variable = (1, 2.0, "three", True)

print(len(tuple_variable))
```

Total values: 4

19 Python Sets

Python Sets

Set is a **Set type data**.

Set is a type of Python Arrays.

Sets are used to **store multiple items** in a single variable.

Set is a **collection** which is ***unordered***, ***unchangeable***, and ***unindexed,*** do ***not allow duplicate values***.

Sets are created using **curly brackets {}** or using **set()** function

using **curly brackets {}** to create **Set**

```
set_variable = {1, 2, 3, 4, 5}

print(set_variable)
```

```
{1, 2, 3, 4, 5}
```

using **set()** function to create Set

```
set_variable = set((1, 2, 3, 4, 5))

print(set_variable)
```

```
{1, 2, 3, 4, 5}
```

Python Sets: len() function

To know **how many items** a Set has, use the **len()** function.
len() function shows **number of items of your Set**

The **name** of Set variable

```
set_variable = {1, 2.0, "three"}

print(len(set_variable))
```

Total values: 3

Python Sets: Join Two Sets, update()

update() method **adds items** from another set into the current set

```
set_1 = {'A', 'B', 'C'}
set_2 = {'D', 'E', 'F'}

# adding items of set_2 into set_1

set_1.update(set_2)
print(set_1)
```

```
{'A', 'E', 'C', 'D', 'F', 'B'}
```

20 Python Dictionary

Python Dictionaries: key:value

Dictionary item is **divided two parts**: 'key' : 'value'

'key' is **key name** of Dictionary Item

Dictionary = {'key' : 'value'}

'value' is the value of Dictionary Item

Python Dictionaries : for loop, search values

If you want to search **values of items**, use **keys** of items as the **index** of **Dictionary:**

```
dictionary_variable = {'name':'Alex', 'age':25}

for d in dictionary_variable:
    print(dictionary_variable[d])
```

```
Alex
25
```

Note: You can use **keys** as index of **Dictionary**, but you ***cannot*** use 0, 1 … etc as index of **Dictionary**

21 Python Functions

Python Functions: Call function

You can **use function**, *how many times* you want by calling **function name**.

If you call **function 1** *time*,
it runs **1** *time*

```
def function():
    print('This is my function')


function()
```

```
This is my function
```

If you call **function 2** *times*,
it runs **2** *times*

```
def function():
    print('This is my function')

function()
function()
```

```
This is my function
This is my function
```

If you call **function 5** *times*,
it runs **5** *times*

```
def function():
    print('This is my function')

function()
function()
function()
function()
function()
```

```
This is my function
This is my function
This is my function
This is my function
This is my function
```

Python Functions: Parameters

Function with **no parameters :**

```
def info()
```

no **parameters**, **empty**

You can write **one** or **many parameters** in the **brackets** after **function name**.

Function with **parameters:**

```
def info(name, age, gender)
```

parameter 1 **parameter** 2 **parameter** 3

comma **comma**

If you write **2** or **more than 1 parameter** in the **brackets** , you have to write **comma ,** after **each parameter**

Python Functions: Parameters and Arguments

Function with **1 parameters** and 1 **argument**:

```
def function(name):

    print(name)

function('Alex')
```

Giving **value** to **parameter**

```
Alex
```

Function with **5 parameters** and 5 **argument**:

```
def function(fname, lname, age, gender, score):

    print(fname)
    print(lname)
    print(age)
    print(gender)
    print(score)

function('Alex', 'Jonh', 25, 'Male', 95.5)
```

Giving **values** to **parameters**

```
Alex
Jonh
25
Male
95.5
```

Note: ***Arguments*** are often shortened to ***args*** in **Python *documentations***

Python Functions: Parameters' Order

When you are declaring **Parameters** in function, first you must write **non default Parameters**, **after that** you write default Parameters

```
def student(age, name = 'Alex')
```

non default Parameters will be written before default Parameters

Python Functions: if, elif, else statements

You can use **if, elif, else** statements in your function:

```
def function(number):

    if number == 0:
        print('number is 0')
    elif number == 1:
        print('number is 1')
    else:
        print('number is not 0 or 1')

function(0)
function(1)
function(2)
function(3)
function(4)
```

```
number is 0
number is 1
number is not 0 or 1
number is not 0 or 1
number is not 0 or 1
```

22 Lambda Function

Python Lambda

Lambda is a function.
Lambda function is a small **unnamed** (***anonymous)*** function.

You can declare **any number of arguments** after **lambda function**

But, Lambda function can have **only one expression**

lambda arguments : **expression**

colon must be written after arguments

Python Lambda: Use lambda inside other function

Examples:

Ex 1)

```
def def_function(a):

    return lambda b: b * a

print(def_function(a = 4)(b = 10))
```

```
40
```

Ex 2)

```
def def_function(a):

    return lambda b: b + a

print(def_function(a = 50)(b = 30))
```

```
80
```

23 Input Function

Python input() function

There is **input() function** in Python, which is used to for user input.

When you use **input() function**, it asks input from user

name variable

```
name = input()
```

If your variable is **equal** to **input() function**, after *running your code*, **input() function** asks you to input **value** for **name** variable

input **value** for **name** variable

```
name = input()
Alex
```

After entering input **value**, please **Enter** button

```
name = input()
```

Output of **name** variable

```
Alex
```

24 Python Classes and Objects

Python Classes and Objects: Create Class

Python is an object oriented programming language.

Object-oriented Programming (OOPs) is a **programming paradigm** (***example***) that uses objects and classes in **programming**.

What is a Class?

Class is the blueprint which is used to create one object or many objects

We can **create** a **class** by using **class keyword**:

Class name

```
class myClass:

    number = 10
```

Colon after **Class name**

number is a **property** of **myClass**, and **it** is equal to **10**

Python Classes and Objects: Modify Object Properties

If you want, you can **modify** Properties on **objects**:

before **modifying**, name='Anna', age=30

```
class Student:

    def __init__(self, name, age):
        self.name = name
        self.age = age

student = Student('Anna', 30)
print(student.name, student.age)

#modifying properties by object
print('----------')

student.name = 'Alex'
student.age = 25
print(student.name, student.age)
```

modify Properties

before **modifying**

```
Anna 30
----------
Alex 25
```

after **modifying**, name='Alex', age=25

Python Classes and Objects: Polymorphism

What is Polymorphism?

Polymorphism is taken from the Greek **words**:

Poly = many
morphism = forms

Polymorphism = many forms

It means that we can **use** the **same function name** for **different targets**.

Example: Cell phone can be Polymorphism, you can **use** it for **many different targets, task, and so on**

Cell phone is Polymorphism

Cell phone can be used

- to take picture
- to talk
- to watch video
- to study

25 Python Inheritance

Python Inheritance: Introduction

What is **Inheritance**?

Inheritance is the ability of **one class** to **derive (inherit)** all methods and properties from **another class**.

There are **two type of classes** in **Python Inheritance**

Parent class

Child class

Parent class is the **class being inherited** from. **Parent class is** also called **Base class** or **Superclass**.

Child class is the **class** that ***inherits from another class***. **Child class is** also called **Derived Class, or Subclass**.

26 Python Scopes

Python Scope: Enclosing Scope, Enclosing Variable

Enclosing Scope is almost similar to **Local** Scope. But, **Local** Scope will be inside of **Enclosing** Scope :

```
def outer():

    name = 'Alex'

    def inner():

        age = 25

        print(name, age)

    inner()

    print(name)
    #print(age)

outer()
```

```
Alex 25
Alex
```

name variable is **local** variable for **outer()** function
But, **it** is **enclosing** variable for **inner()** function. Therefore, you can use it inside **inner()** function

age variable is **local** variable for **inner()** function

Blue rectangle is **Local** scope and it is the **region** to **use** age variable

Red rectangle **is** **Enclosing scope** and **it** is the **region** to **use** name variable.

Python Scope: Global Keyword

global keyword is used to **convert** local variable or enclosing variable to **global** variable

```
def student():
    global name
    name = 'Alex'
print(name)
```

```
Alex
```

global keyword makes name variable to be global variable

name variable can be used in anywhere, because **it** is **global variable**

27 Python Error Handling

Python Try Except: Many Exceptions

If you want, you can use **multiple** except **blocks** for handling error.

Also you can use the **name** of **occurred** error with except **block:**

```
print(name)

---------------------------------------
NameError
<ipython-input-1-9ba126b17b03> in <mod
----> 1 print(name)

NameError: name 'name' is not defined
```

Using **NameError** with except **block**

1st except **block**

2nd except **block**

```
try:
    print(age)

except NameError:
    print('age is not declared')

except:
    print('Error is occured, please check your code')
```

1st except **block** handled error, therefore **output** is → `age is not declared`

28 Python Modules

Python Modules

What is a **Module**?

Module is the **code library** of Python

Module is just **.py** (python) **file**.

Module can be **used** through **other .py** (python) **file**

There are **3 kind** of **Modules in Python**

Your **own Modules**

Your own python **libraries,** which are created by yourself

Built-in Modules
(Internal **Modules**)

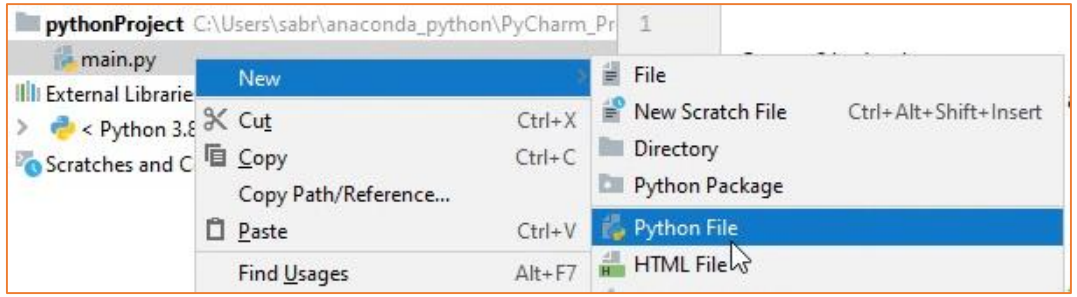
existed python **libraries,** and they are **already installed** when you **installed python**

External Modules

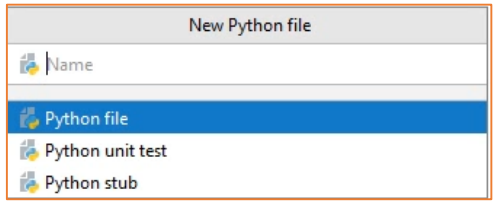
existed python **libraries,** But you have to **install** them **to use**.

Python Modules: Create your own Module

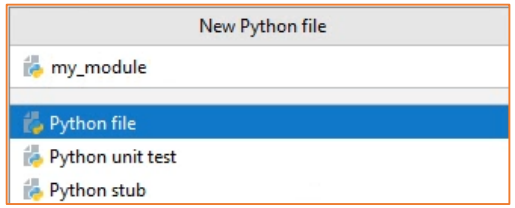
1. Create new **.py** file. Go **pythonProject** or main.py and crick **right side** of your **mouse** on **pythonProject** or main.py and **New** and click **Python File**



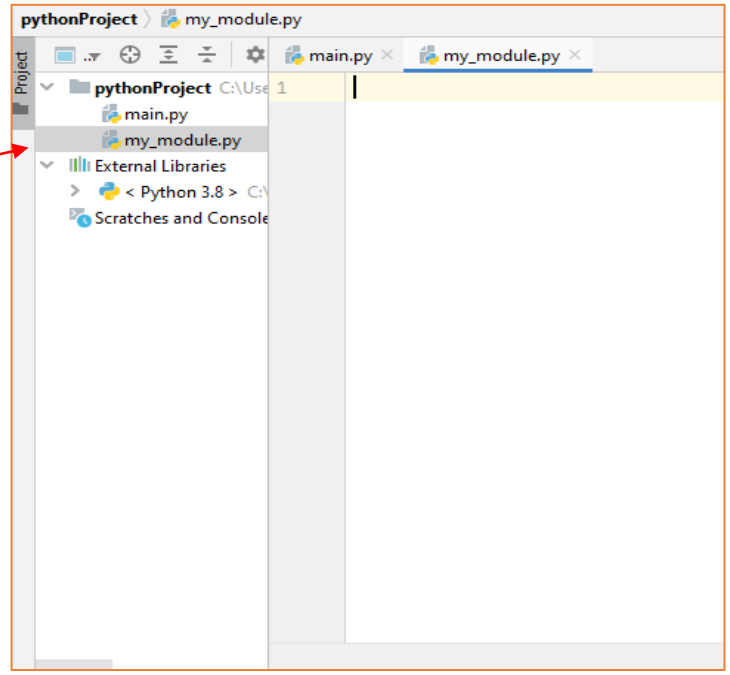
2. Give **name** to Python file



3. **name** is **my_module. Choose Python File**



4. my_module.py file is **created,**
That means **you created your own module.**
You can **write** your **codes** inside it



Python Modules: Renaming Module

import keyword used to **load** or **call** existed modules.

When you are **importing a module**, you can rename it by using **as keyword**

```
#rename module
import my_module as mm

student = mm.Student('Alex', 25, 'Male')
student.show_info()
```

After running, output:

```
Student name =  Alex
Student age =  25
Student gender =  Male
```

```
#rename module
import my_module as own_module

student = own_module.Student('Alex', 25, 'Male')
student.show_info()
```

After running, output:

```
Student name =  Alex
Student age =  25
Student gender =  Male
```

```
#rename module
import my_module as person

student = person.Student('Alex', 25, 'Male')
student.show_info()
```

After running, output:

```
Student name =  Alex
Student age =  25
Student gender =  Male
```

Python Modules: External Modules

What are External Modules?

External Modules are also the **code library** of **Python**, but you **have to install them** to **use**.

You can install **External Modules**, by using **their package names**

Module name	Package name	Import Module *after installation*	Installing by pip or conda
numpy	numpy	import numpy	pip install numpy or conda install numpy
cv2	opencv-python	import cv2	pip install opencv-python
pandas	pandas	import pandas	pip install pandas or conda install pandas

Python Modules: Installing External Modules

For example, you want to install numpy **package**, for *installing it* you can use:

Anaconda Prompt

using pip:

```
Anaconda Prompt (anaconda3)
(base) C:\Users\sabr>pip install numpy
```

using conda:

```
Select Anaconda Prompt (anaconda3)
(base) C:\Users\sabr>conda install numpy
```

Pycharm Terminal

using pip:

```
> pip install numpy
TODO  Problems  Terminal
```

using conda:

```
> conda install numpy
TODO  Problems  Terminal
```

Jupyter Notebook

using pip, type ! before pip

```
!pip install numpy
```

using conda, type ! before conda

```
!conda install numpy
```